AI LAB ASSESSMENT 3

Name: K. Manoj Kumar
Register Number:23MIS0159
Course code: ISWE302P

1. Considering the following small dataset on housing prices, apply Linear Regression algorithm to predict a continuous value (e.g., housing prices) based on features in the dataset.

Size (sq ft)	Bedrooms	Price
1500	3	300000
2000	4	400000
2500	4	500000
1800	3	350000
2200	4	450000
1700	3	325000
2100	4	420000
1600	3	310000
2400	5	490000
1900	3	370000

(Instructions:

- Install required libraries → pip install pandas
- Store the above dataset in housing_prices.xlsx and place it in the same directory as your Python script
- Run the Python Code:
  - To load the Excel file
  - Preprocess the data
  - Train a linear regression model
  - Display the Mean Squared Error and a plot comparing actual vs. predicted housing prices.

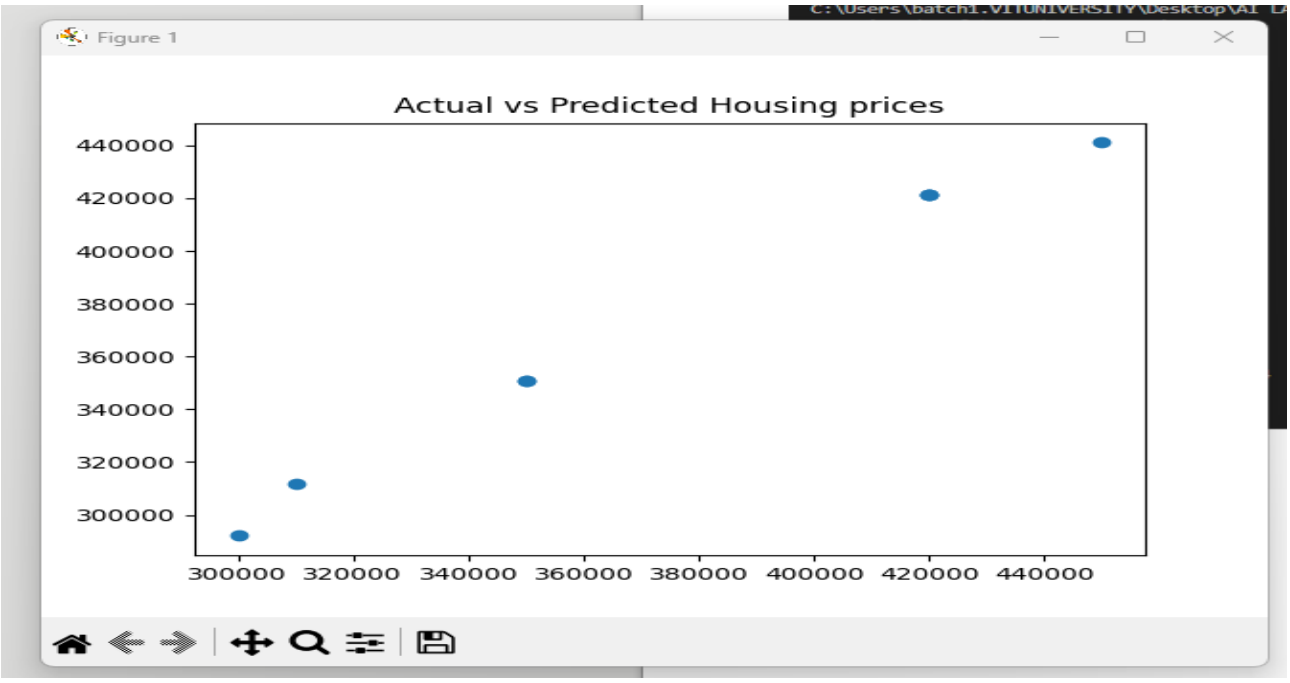
CODE:

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error


data=pd.read_excel("housing_prices.xlsx")

print(data)

X=data[['Size (sq ft)','Bedrooms']]

y=data['Price']

X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=42)

model=LinearRegression()
model.fit(X_train,y_train)

y_pred=model.predict(X_test)
mse=mean_squared_error(y_test,y_pred)
print("Mean sqayre Error ",mse)

print('\n Model co efficient :')
print("Size co efficient :",model.coef_[0])
```

```
print("Bedrooms coefficient :",model.coef_[1])
print("Intercept:",model.intercept_)


plt.scatter(y_test,y_pred)
plt.xlabel=("Actual price")
plt.ylabel=("Predicted price")
plt.title("Actual vs Predicted Housing prices")

plt.show()
```

OUTPUT:

```
C:\Users\batch1.VITUNIVERSITY\Desktop\AI LAB\Jenkins-admin\Assessment 3>python linear_regression.py
    Size (sq ft)  Bedrooms   Price
0           1500         3  300000
1           2000         4  400000
2           2500         4  500000
3           1800         3  350000
4           2200         4  450000
5           1700         3  325000
6           2100         4  420000
7           1600         3  310000
8           2400         5  490000
9           1900         3  370000
33          1800         3  350000
34          2200         4  450000
35          1700         3  325000
36          2100         4  420000
37          1600         3  310000
38          2400         5  490000
Mean sqayre Error  19096350.907546163

 Model co efficient :
Size co efficient : 195.7266769876785
Bedrooms coefficient : 12002.629737238354
Intercept: -37504.294530570915
```



2. Considering the following small dataset on Spam classification, classify instances into binary categories (e.g., spam or not spam) using Logistic Regression.

Email Length	Has Link	Is Spam (0 or 1)
50	1	0
300	1	1
200	0	0
400	1	1
150	0	0
350	1	1
100	0	0
500	1	1
250	0	0
450	1	1

(Instructions:

- Load the Excel file using pandas.
- Perform data preprocessing (e.g., encode categorical variables).
- Split the data into training and testing sets.
- Train a Logistic Regression model.
- Evaluate the model with accuracy, precision, and recall.)

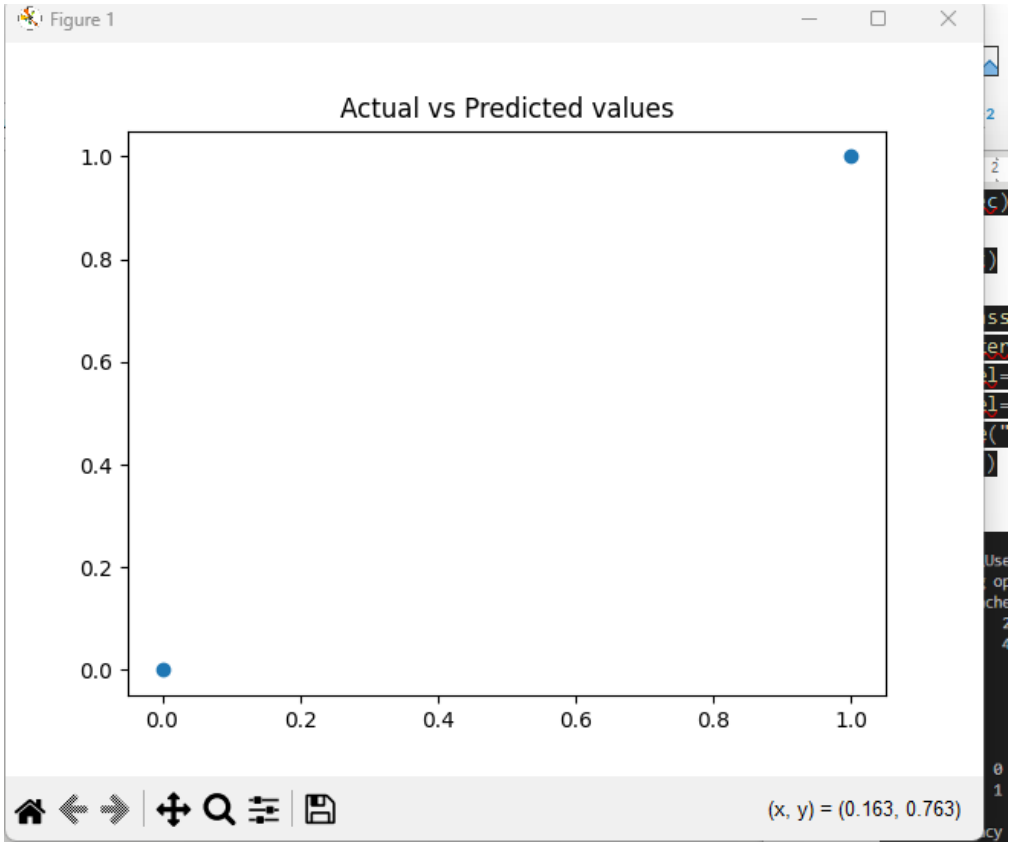
CODE:

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import
accuracy_score,precision_score,recall_score,classification_report


data=pd.read_excel("spam_data.xlsx")

print(data)

X=data[['Email Length','Has Link']]

y=data['is Spam']

X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=42)

model=LogisticRegression()
model.fit(X_train,y_train)

y_pred=model.predict(X_test)


acc=accuracy_score(y_test,y_pred)
prec=precision_score(y_test,y_pred)
```

```
rec=recall_score(y_test,y_pred)

print(acc)

print(prec)

print(rec)

print(classification_report(y_test,y_pred))
plt.scatter(y_test,y_pred)
plt.xlabel=("Actual pred")
plt.ylabel=("Predicted prd")
plt.title("Actual vs Predicted values")
plt.show()
```

OUTPUT:

```
(venv) C:\Users\batch1.VITUNIVERSITY\Desktop\AI LAB\Jenkins-admin\Assessment 3>pip install openpyxl
Collecting openpyxl
  Using cached openpyxl-3.1.5-py2.py3-none-any.whl (250 kB)
8          250        0        0
9          450        1        1
1.0
1.0
1.0
              precision    recall  f1-score   support

           0       1.00      1.00      1.00         1
           1       1.00      1.00      1.00         1

    accuracy                           1.00         2
   macro avg       1.00      1.00      1.00         2
weighted avg       1.00      1.00      1.00         2
```



3. Considering the following small dataset, Group data points into clusters based on their features using K-Means Clustering.

Income	Age	Spending Score
40000	22	65
50000	25	55
60000	30	50
70000	35	40
80000	40	45
30000	20	70
90000	45	30
100000	50	20
110000	55	15
120000	60	10

(Instructions:

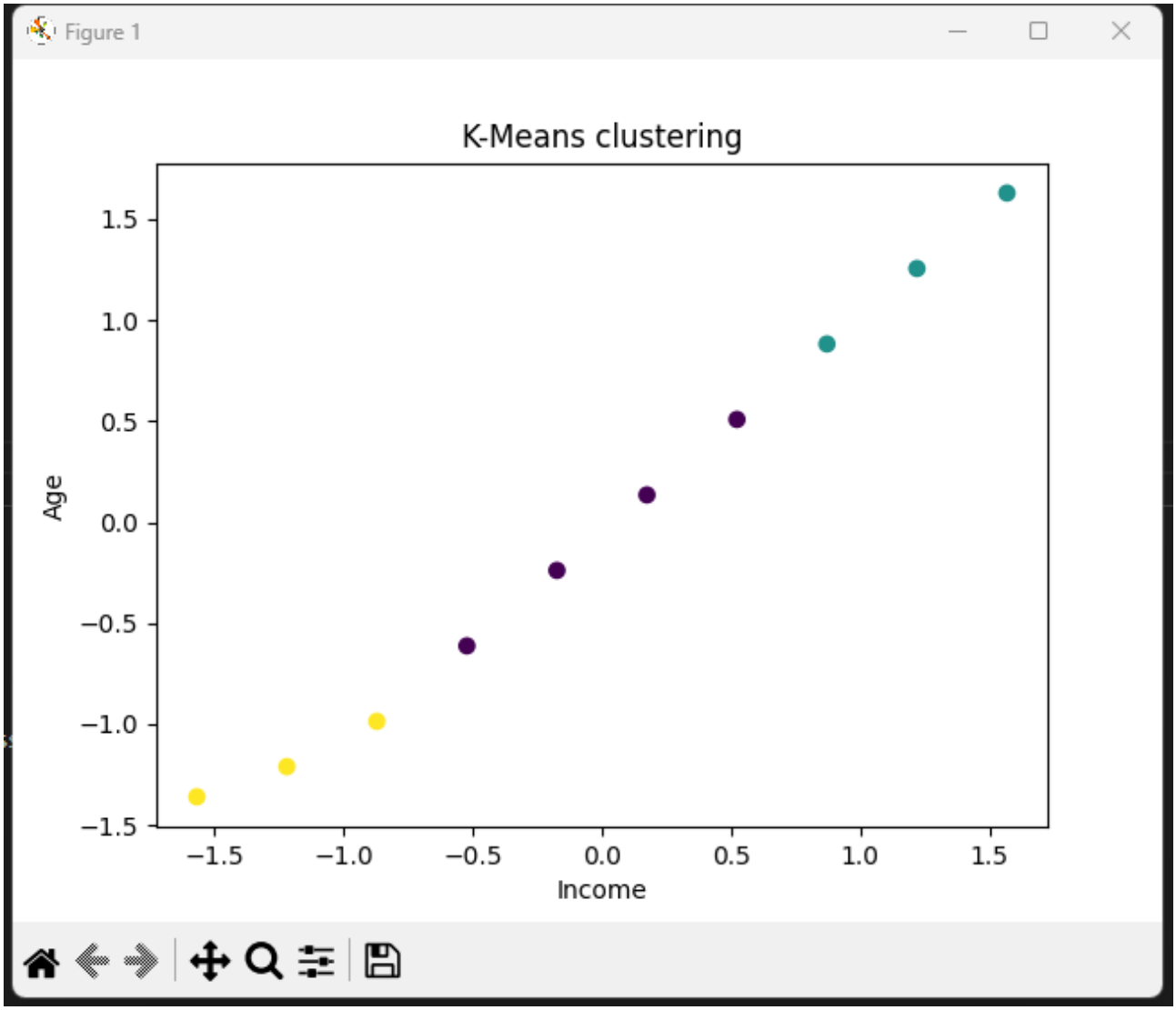
- Load the Excel file using pandas.
- Preprocess the data (e.g., scaling features).
- Apply K-Means Clustering to identify clusters in the dataset.
- Visualize the clusters using a scatter plot.)

CODE:

```
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

df=pd.read_excel("Spending_score.xlsx")

print(df.head())

X=df.select_dtypes(include=['float64','int64'])

scaler=StandardScaler()
X_Scaled=scaler.fit_transform(X)

k=3
kmeans=KMeans(n_clusters=k,random_state=42)

cluster=kmeans.fit_predict(X_Scaled)

df['Clusters']=cluster

print(df)

plt.scatter(X_Scaled[:,0],
       X_Scaled[:,1],c=cluster)
plt.xlabel("Income")
plt.ylabel("Age")
plt.title("K-Means clustering")
plt.show()
```

OUTPUT:

```
(venv) C:\Users\batch1.VITUNIVERSITY\Desktop\AI LAB\Jenkins-admin\Assessment 3>python k_means.py
   Income  Age  SpendingScore
0   40000   22             65
1   50000   25             55
2   60000   30             50
3   70000   35             40
4   80000   40             45
   Income  Age  SpendingScore  Clusters
0   40000   22             65         2
1   50000   25             55         2
2   60000   30             50         0
3   70000   35             40         0
4   80000   40             45         0
5   30000   20             70         2
6   90000   45             30         0
7  100000   50             20         1
8  110000   55             15         1
9  120000   60             10         1
```



4. Considering the following small dataset, classify instances into multiple categories (e.g., species of flowers) using Decision Tree Classifier

Petal Length	Petal Width	Species
1.4	0.2	Setosa
1.3	0.2	Setosa
4.7	1.4	Versicolor
4.5	1.5	Versicolor
5.5	2.1	Virginica
5.1	1.9	Virginica
4.9	1.8	Virginica
1.6	0.4	Setosa
4.2	1.3	Versicolor
5	1.5	Versicolor

(Instructins:

- Load the Excel file using pandas.
- Preprocess the data.
- Split the data into training and testing sets.
- Train a Decision Tree classifier.
- Visualize the tree and evaluate the model's accuracy.)

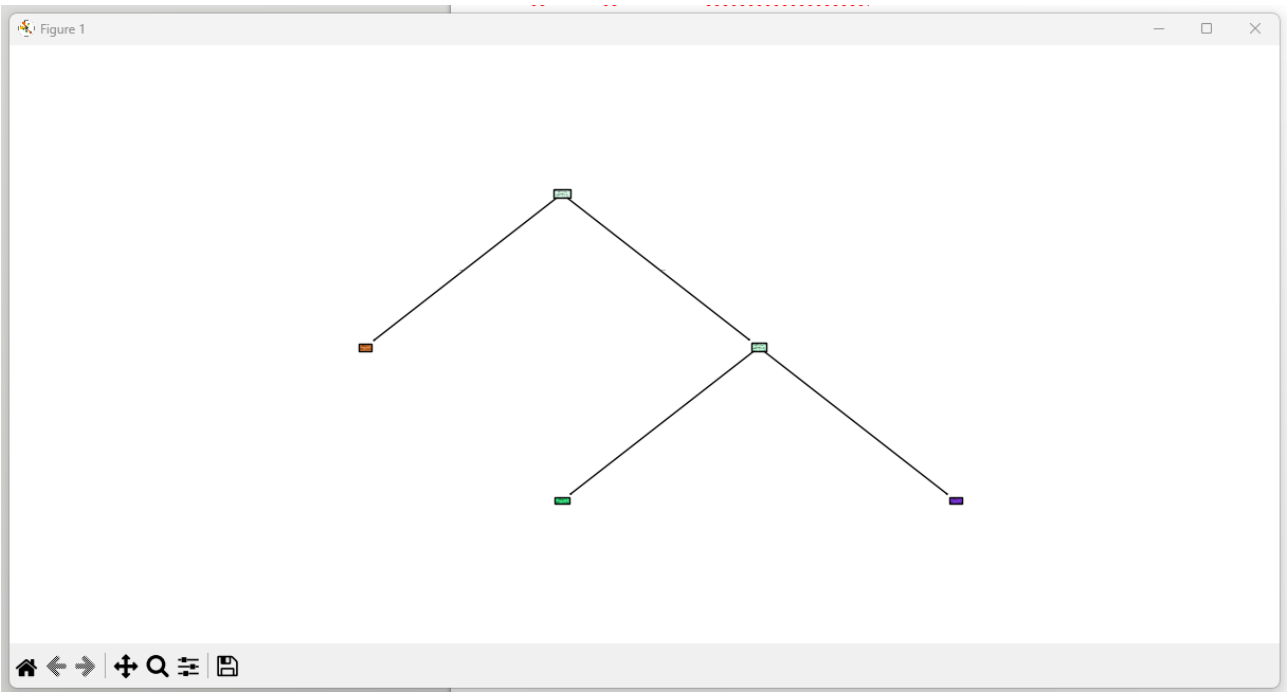
CODE:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier,plot_tree
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt

df=pd.read_excel('Petal_dataset.xlsx')

X=df[['PetalLength','Petalwidth']]
y=df['Species']

encoder=LabelEncoder()
y_encoded = encoder.fit_transform(y)

X_train,X_test,y_train,y_test =
train_test_split(X,y_encoded,test_size=0.3,random_state=42)

model=DecisionTreeClassifier()
model.fit(X_train,y_train)

y_pred=model.predict(X_test)

accuracy_score=accuracy_score(y_test,y_pred)
print('Accuracy :',accuracy_score)

plt.figure(figsize=(12,0))
```

```
plot_tree(model,
      feature_names=['PetalLength','PetalWidth'],
      class_names=encoder.classes_,
      filled=True)

plt.show()
```

OUTPUT:

```
(venv) C:\Users\batch1.VITUNIVERSITY\Desktop\AI LAB\Jenkins-admin\Assessment 3>python Decision_tree.py
Accuracy : 1.0
```



5. Considering the following small dataset, predict a continuous value using an ensemble of decision trees using Random Forest Regression

Date	Temperature (°C)	Humidity (%)	Energy Consumption (kWh)
01-09-2024	22	60	30
02-09-2024	24	55	32
03-09-2024	23	58	31
04-09-2024	25	50	29
05-09-2024	26	52	34

(Instructions:

- Load the Excel file using pandas.
- Preprocess the data.
- Split the data into training and testing sets.
- Train a Random Forest model.
- Evaluate the model performance using metrics like R^2.)

CODE:

```
import pandas  as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import r2_score

df=pd.read_excel('EnergyConsumption_data.xlsx')
df['Date']=pd.to_datetime(df['Date'])

df['Date']=df['Date'].map(pd.Timestamp.toordinal)


X=df[['Date','Temperature','Humidity']]
y=df['EnergyConsumption']

X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.3,random_state=42)

model = RandomForestRegressor(n_estimators=100,
                              random_state=42)
model.fit(X_train,y_train)

y_pred=model.predict(X_test)

r2=r2_score(y_test,y_pred)
print('R2 score :',r2)

print("Actual :",list(y_test))
```

```
print("Predicted :",list(y_pred))
```

OUTPUT:

```
(venv) C:\Users\batch1.VITUNIVERSITY\Desktop\AI LAB\Jenkins-admin\Assessment 3>python RandomForest.py
R2 score : -9.600399999999999
Actual : [32, 34]
Predicted : [np.float64(30.58), np.float64(29.62)]
```